

LINUX THEORY ASSIGNMENT 3

NAME	SIVARAMAKRISHNAN R
ROLL NO	261100690026
BRANCH	CYBER SECURITY

Linux Filesystem — Lab Assignment

Exercise 1 – Identify Your Linux Filesystem

Commands used: lsblk, lsblk -f, df -T, df -h

```
stvagilinux:~$ df -T
Filesystem      Type 1K-blocks    Used Available Use% Mounted on
tmpfs           tmpfs 695680      2132    693736    1% /run
/dev/sda2       ext4 25623780 10852808 13444024  45% /
tmpfs           tmpfs 1739660      0    1739660    0% /dev/shm
tmpfs           tmpfs 1739664      0    1739656    1% /tmp
none           tmpfs 1024         0      1024    0% /run/credentials/systemd-journald.service
none           tmpfs 1024         0      1024    0% /run/credentials/systemd-resolved.service
tmpfs           tmpfs 347932       0      347948    1% /run/user/1000
stvagilinux:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           680M  2.1M  678M   1% /run
/dev/sda2       25G   11G   13G   45% /
tmpfs           1.7G   0   1.7G   0% /dev/shm
tmpfs           1.7G 8.0K 1.7G   1% /tmp
none           1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
none           1.0M   0   1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           340M 84K  340M   1% /run/user/1000
stvagilinux:~$ df -T /
Filesystem      Type 1K-blocks    Used Available Use% Mounted on
/dev/sda2       ext4 25623780 10852808 13444024  45% /
stvagilinux:~$
```

Findings table:

Device/Partition	Filesystem Type	Size	Mount Point
/dev/vda	ext4	252G	/
tmpfs	tmpfs	4.0G	/dev/shm
tmpfs	tmpfs	2.0G	/sys/fs/cgroup
rclone-filestore	fuse.rclone	1.0P (virtual)	/mnt/user-data/*, /mnt/transcripts

Notes: This machine has a single primary block device, /dev/vda, which is not split into separate numbered partitions — it is mounted directly as the ext4 root filesystem. tmpfs entries are RAM-backed virtual filesystems, and the rclone-filestore entries are network/FUSE mounts used for shared storage, not physical disk partitions.

Exercise 2 – Disk → Partition → Filesystem → Mount Point

Relationship: A disk is the physical (or virtual) storage device; it can be divided into one or more partitions, each a logically separate section of the disk. A filesystem (such as ext4) is the data structure written onto a partition that organizes how files and directories are stored, named, and tracked. The mount point is the directory in the existing directory tree where that filesystem is attached, making its contents accessible through normal path names. In short: Disk holds Partitions, Partitions are formatted with a Filesystem, and a Filesystem is made reachable via a Mount Point.

Exercise 3 – Explore the Root Filesystem

```
siva@linux:~$ ls
1.txt  Desktop  Documents  Music  Public  Videos  crypto  linux_fileinst  test.txt
siva@linux:~$ cd /
siva@linux:/$ pwd
/
siva@linux:/$ ls
bin  boot  cdrom  dev  etc  home  lib  lib64  lost+found  media  mnt  opt  proc  root  run  sbin  snap  srv
siva@linux:/$ ls -d /*
/bin /boot /cdrom /dev /etc /home /lib /lib64 /lost+found /media /mnt /opt /proc /root /run /sbin /snap /srv
siva@linux:/$ ls -la
total 80
drwxr-xr-x 20 root root 4096 Aug  5 13:19 .
drwxr-xr-x 20 root root 4096 Aug  5 13:19 ..
lrwxrwxrwx 1 root root   7 Apr 20 08:46 bin -> usr/bin
drwxr-xr-x 3 root root 4096 Aug 25 04:20 boot
dr-xr-xr-x 2 root root 4096 Aug  5 13:14 cdrom
drwxr-xr-x 19 root root 4380 Sep  7 19:01 dev
drwxr-xr-x 143 root root 12288 Sep  3 12:34 etc
drwxr-xr-x 7 root root 4096 Sep  1 04:36 home
lrwxrwxrwx 1 root root   7 Apr 20 08:46 lib -> usr/lib
lrwxrwxrwx 1 root root   9 Apr 20 08:46 lib64 -> usr/lib64
drwx----- 2 root root 16384 Aug  5 13:21 lost+found
drwxr-xr-x 2 root root 4096 Apr 23 01:15 media
drwxr-xr-x 2 root root 4096 Apr 23 01:15 mnt
drwxr-xr-x 3 root root 4096 Aug 19 13:33 opt
dr-xr-xr-x 329 root root   0 Sep  7 19:01 proc
drwx----- 6 root root 4096 Aug 19 13:56 root
drwxr-xr-x 45 root root 1040 Sep  7 19:01 run
lrwxrwxrwx 1 root root   8 Apr 20 08:46 sbin -> usr/sbin
drwxr-xr-x 17 root root 4096 Aug 10 14:20 snap
drwxr-xr-x 2 root root 4096 Apr 23 01:15 srv
dr-xr-xr-x 13 root root   0 Sep  7 19:01 sys
drwxrwxrwt 16 root root 360 Sep  7 19:02 tmp
drwxr-xr-x 12 root root 4096 Apr 23 01:15 usr
drwxr-xr-x 15 root root 4096 Sep  1 07:02 var
siva@linux:/$
```

1. Current working directory before navigating: /home/claude/work
2. Navigated to root with cd /.
3. Contents of / listed above (23 entries).
4. Directories directly under /: bin, boot, dev, etc, home, lib, lib64, media, mnt, opt, proc, root, run, sbin, srv, sys, tmp, usr, var (plus environment-specific extras like lost+found, old_root, bin.usr-is-merged, lib.usr-is-merged, sbin.usr-is-merged).
5. Hidden entries: only . (current directory) and .. (parent directory) are hidden at /; there was one visible non-directory file, container_info.json, which is not hidden.

Exercise 4 – Linux Filesystem Hierarchy Investigation

Directory	Example Contents	Purpose
/home	claudio/, ubuntu/	Holds personal directories for regular (non-root) users; each user gets a subfolder for their own files and settings.
/root	.bashrc, .config, .profile, .ssh	The home directory of the root (administrator) user, separate from /home for security and boot-time reasons.
/etc	passwd, hosts, hostname, ImageMagick-6/	System-wide configuration files for the OS and installed applications; mostly plain text, editable by root.
/var	log/, cache/, lib/, backups/	Variable data that changes while the system runs — logs, caches, spool files, application state.
/var/log	dpkg.log, alternatives.log, journal/, wtmp	Stores system and application log files used for monitoring, troubleshooting, and forensic investigation.
/tmp	hsperfdata_root/, phantomjs/, various lock files	Temporary storage for short-lived files created by users and programs; world-writable and usually cleared on reboot.
/usr	bin/, lib/, include/, share/	Secondary hierarchy holding the bulk of installed user programs, libraries, and shared/read-only data.
/boot	(kernel/bootloader files when present)	Holds files needed to boot the system, such as the kernel image and bootloader configuration.
/dev	null, zero, sda, tty, cpu/	Device files representing hardware and virtual devices; lets programs access devices as if they were files.
/proc	cpuinfo, meminfo, numbered PID folders (1, 10, 599...)	A virtual filesystem exposing live kernel and process information; has no real content on disk.

Exercise 5 – Find Your Home Directory

```
/home/siva
siva@linux:/$ cd "$HOME"
siva@linux:~$ pwd
/home/siva
siva@linux:~$ ls
1.txt  Desktop  Downloads  Music  Pictures  Public  Videos  crypto  linux_fileinst  test.txt
siva@linux:~$ ls -la
total 120
drwxr-x--- 20 siva siva 4096 Sep  4 14:58 .
drwxr-xr-x  7 root root 4096 Sep  1 04:36 ..
-rw-r----- 1 siva siva 9447 Sep  4 14:01 .bash_history
-rw-r--r--  1 siva siva 220 Feb 13 2026 .bash_logout
-rw-r--r--  1 siva siva 3771 Feb 13 2026 .bashrc
drwx----- 15 siva siva 4096 Aug 11 13:34 .cache
drwx----- 17 siva siva 4096 Aug 11 13:58 .config
-rw-rw-r--  1 siva siva  77 Aug 11 14:51 .gitconfig
drwx-----  2 siva siva 4096 Sep  4 13:33 .gnupg
drwx-----  4 siva siva 4096 Aug  5 14:18 .local
-rw-r--r--  1 siva siva 807 Feb 13 2026 .profile
drwx-----  2 siva siva 4096 Aug  5 14:18 .ssh
-rw-r-----  1 siva siva 12017 Sep  4 14:58 .viminfo
-rw-rw-r--  1 siva siva  8 Aug 21 12:21 1.txt
drwxr-xr-x  6 siva siva 4096 Sep  3 07:03 Desktop
drwxr-xr-x  5 siva siva 4096 Sep  4 09:47 Documents
drwxr-xr-x  7 siva siva 4096 Sep  4 13:08 Downloads
drwxr-xr-x  2 siva siva 4096 Aug  5 14:18 Music
drwxr-xr-x  3 siva siva 4096 Aug 10 14:02 Pictures
drwxr-xr-x  2 siva siva 4096 Aug  5 14:18 Public
drwxr-xr-x  2 siva siva 4096 Aug  5 14:18 Templates
drwxr-xr-x  2 siva siva 4096 Aug  5 14:18 Videos
-rw-rw-r--  1 siva siva  8 Aug 21 12:21 aaa.txt
drwxrwxr-x  2 siva siva 4096 Aug 19 12:34 crypto
drwxrwxr-x  3 siva siva 4096 Aug 11 14:55 linux05-lab-theory
drwxrwxr-x  2 siva siva 4096 Sep  4 10:28 linux_fileinst
-rw-r-----  7 siva siva 4096 Aug 11 14:24 snap
-rw-rw-r--  1 siva siva  8 Aug 21 12:21 test.txt
drwxrwxr-x  2 siva siva 4096 Aug 11 14:12 windows_share
```

1. Username: root
2. Home directory: /root (found via \$HOME, not assumed)
3. Navigated there with `cd ~`.
4. Absolute path confirmed with `pwd`: /root
5. Normal `ls` showed no visible files.
6. `ls -a` revealed 4 hidden entries: `.bashrc`, `.config`, `.profile`, `.ssh`.

Exercise 6 – Hidden Files Investigation

Comparing `ls` and `ls -a` in /root shows that `ls -a` reveals four extra entries not shown by plain `ls`: `.bashrc`, `.profile`, `.config`, and `.ssh` (plus `.` and `..`).

1. Linux identifies a hidden file purely by its name starting with a dot (`.`) — there is no separate hidden-file attribute or flag like in Windows.
2. A normal `ls` does not display dotfiles because the tool is designed to filter out entries beginning with `'.'` by default, keeping directory listings uncluttered with configuration/state files.
3. `'.'` refers to the current directory itself, and `'..'` refers to its parent directory; both are hidden by the same dot-name convention.

4. Yes — a hidden file can be accessed normally (opened, read, executed, edited) if its exact name/path is known; hiding only affects default listing visibility, not permissions or access.
5. From a cybersecurity perspective, hidden files should not be assumed safe or malicious purely because of the dot prefix: legitimate configuration files (.bashrc, .ssh) are hidden by convention, but attackers also frequently use dot-prefixed names to conceal malware, backdoors, or exfiltrated data in plain sight, so each file needs to be evaluated on its content, origin, and permissions rather than its naming alone.

Exercise 7 – Build a Small Filesystem Hierarchy

Command used to create the whole hierarchy in a single call:

Verification with tree:

```
siva@linux:~$ tree Desktop
Desktop
├── 1.out
├── a.txt
├── error.sh
├── error.txt
├── file1.txt
├── file_hard.txt
├── fileprc2.txt
├── fileprc4.txt -> fileprc3.txt
├── n.txt
├── owner_lab
│   ├── credentials.txt
│   ├── incident.txt
│   └── network.log
├── payload.sh
├── permission_lab
│   ├── sample_dir
│   │   └── test.txt
│   └── sample.txt
├── sed
│   └── dum.txt
├── siva@pract1
│   └── siva@pract2
├── tra.txt
├── y.txt
└── ytxt
```

The hierarchy and all five required files were created and verified successfully.

Exercise 8 – Absolute and Relative Paths

An absolute path always starts from the root of the filesystem (/) and fully specifies a location regardless of the current directory, e.g. /etc or /root/cyber_lab/reports.

A relative path is interpreted starting from the current working directory, e.g. running cd evidence/network while already inside cyber_lab.

The difference between / and ~ is that / is the absolute root of the entire filesystem tree, the top of everything, while ~ is a shorthand that expands to the current user's home directory (here, /root) — a fixed, user-specific absolute path, not the filesystem root.

Exercise 9 – Identify File Types

Object	Type
/etc/passwd	Regular text file (user account database)
/etc	Directory
/bin/bash	Regular file — ELF executable binary (compiled program)
/dev/null	Character special (device) file
/proc	Directory (virtual/pseudo filesystem)
security.log	Regular file (empty, plain text log)
check.sh	Regular file (empty shell script)

```
siva@linux:~$ file /etc/passwd
/etc/passwd: ASCII text
siva@linux:~$ file /etc
/etc: directory
siva@linux:~$ file /bin/bash
/bin/bash: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld
b679fb438b940fc17e2e4951ac52ac89, for GNU/Linux 3.2.0, stripped
siva@linux:~$ file /bin/bash
/bin/bash: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld
b679fb438b940fc17e2e4951ac52ac89, for GNU/Linux 3.2.0, stripped
siva@linux:~$ file /dev/null
/dev/null: character special (1/3)
siva@linux:~$ file /proc
/proc: directory
siva@linux:~$ file ~/Desktop/error.sh
/home/siva/Desktop/error.sh: a usr/bin/bash script, ASCII text executable
```

Exercise 10 – Examine File Metadata

Attribute	Value
Filename	investigation.txt
File size	100 bytes
File type	Regular file
Owner	root
Group	root
Inode number	573474
Access time (atime)	2026-08-28 12:44:33
Modification time (mtime)	2026-08-28 12:44:55
Change time (ctime)	2026-08-28 12:44:55

1. Metadata is data about the file — properties such as size, ownership, permissions, timestamps, and inode number — rather than the file's actual data.
2. No, metadata is not the same as file content: content is the bytes stored inside the file (what you'd see with `cat`), while metadata describes attributes of the file itself, tracked separately by the filesystem.
3. The modification time (`mtime`) changes when the file's contents are altered; the change time (`ctime`) also updates whenever `mtime` does, because `ctime` reflects metadata changes generally, and editing content is one such change.

Exercise 11 – Explore /etc

```
sivaglinux:~/Desktop/owner_lab$ ls -l /etc/passwd
-rw-r--r-- 1 root root 3058 Sep  3 07:01 /etc/passwd
sivaglinux:~/Desktop/owner_lab$ ls -l /etc/group
-rw-r--r-- 1 root root 1244 Sep  3 11:20 /etc/group
sivaglinux:~/Desktop/owner_lab$ ls -l /etc/hostname
-rw-r--r-- 1 root root 6 Aug  5 14:18 /etc/hostname
sivaglinux:~/Desktop/owner_lab$ ls -l /etc/hosts
-rw-r--r-- 1 root root 273 Feb 10 2026 /etc/hosts
```

1. /etc primarily stores system-wide configuration files: settings for the OS, services, and installed applications, mostly in plain text.
2. /etc/passwd contains the local user account database — one line per user with username, UID, GID, home directory, and login shell (password hashes now live in /etc/shadow instead).
3. /etc/hostname contains the system's configured hostname, here 'vm'.
4. /etc is security-sensitive because it defines how the system authenticates users, what services run and how, and other core behavior; tampering with files here (e.g. adding a user, changing sudoers, altering hosts) can grant unauthorized access or redirect traffic, so its contents are a prime target and a prime source of forensic evidence.

Exercise 12 – Explore Linux Log Storage

1. Log files: alternatives.log, bootstrap.log, dpkg.log, fontconfig.log, faillog, lastlog, wtmp, btmp.
2. Log directories: apt/, journal/, private/.
3. Recently modified entries: dpkg.log and the apt/ directory, both touched most recently in this session.
4. Readable entries as a normal (non-root) user: alternatives.log, dpkg.log, fontconfig.log, README, lastlog, wtmp are world- or group-readable.
5. Restricted entries: btmp (rw-rw---- owned by root:utmp) and the private/ directory (drwx-----, root only) are not readable by an ordinary user.

Why does Linux maintain logs? To record system and application events over time — startups, package changes, authentication attempts, errors — so administrators can monitor health and diagnose problems after the fact.

Why are logs useful during incident investigation? They provide a timestamped trail of what happened, when, and by which process/user, letting investigators reconstruct the sequence of an attack or fault.

Why might access to some logs be restricted? Logs such as `btmpt` (failed login attempts) and auth-related logs can reveal usernames, login patterns, or other sensitive details that could help an attacker, so read access is limited to root or specific groups like `utmp`.

Exercise 13 – Investigate `/tmp`

1. Files/directories currently exist: yes — several directories (`hsperfdata_root`, `node-compile-cache`, `phantomjs`, `rlone-mounts`) and multiple lock/config files.
2. Hidden files: none were found directly under `/tmp` in this listing (no dot-prefixed entries besides `.` and `..`).
3. Ownership: everything present is owned by `root:root` in this environment.
4. Recently modified entries: `rlone-mount-config.json` and `rlone-mounts/` show the most recent timestamps, matching the current session's activity.

Why can temporary directories be relevant during a cybersecurity investigation? `/tmp` is world-writable by design, so it is a common location for attackers to drop staged payloads, downloaded tools, or exfiltration data because any user or process can write there without special privileges; its contents (and their owners/timestamps) can reveal what a compromised process or user was doing.

Exercise 14 – Explore the `/proc` Filesystem

1. `/proc/cpuinfo` provides live details about the CPU(s): vendor, model name, clock speed, cache size, core/thread counts, and feature flags.
2. `/proc/meminfo` provides live memory statistics: total, free, and available RAM, buffers, cache, and swap usage.
3. Numbered directories under `/proc` correspond to the PIDs (process IDs) of every currently running process, exposing that process's runtime information (command line, memory maps, open files, status, etc.).
4. No, `/proc` is not an ordinary disk directory — it is a virtual (pseudo) filesystem generated by the kernel in memory on the fly; nothing under it is stored on disk, and its size shows as 0 bytes.
5. `/proc` is useful during a security investigation because it offers a real-time view into running processes, open network connections, loaded kernel modules, and resource usage — letting an investigator spot a malicious or hidden process while it is still active, before it can be cleaned up or before a reboot erases the evidence.

Exercise 15 – Explore /dev

1. /dev represents devices (hardware and virtual/pseudo-devices) exposed through the filesystem, so programs can interact with them using standard file operations (open, read, write).

```
siva@linux:~/Desktop/owner_lab$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Sep  7 19:01 /dev/null
siva@linux:~/Desktop/owner_lab$ ls -l /dev/zero
crw-rw-rw- 1 root root 1, 5 Sep  7 19:01 /dev/zero
siva@linux:~/Desktop/owner_lab$ file /dev/null
/dev/null: character special (1/3)
siva@linux:~/Desktop/owner_lab$ file /dev/zero
/dev/zero: character special (1/5)
siva@linux:~/Desktop/owner_lab$
```

2. No, entries in /dev are not ordinary user files — they are special device files (character or block) that the kernel routes to a device driver rather than to disk-stored data.
3. /dev/null is a character special file (type 'c' in the permissions, major/minor numbers 1,3), meaning it transfers data byte-by-byte rather than in fixed blocks.
4. Linux exposes devices through the filesystem to keep a unified, consistent interface — the same tools and system calls used for regular files (open, read, write, close) also work on hardware devices, following the Unix philosophy of 'everything is a file'.

Exercise 16 – Filesystem Space Investigation

1. Total filesystem capacity: 252 GB (on /dev/vda, mounted at /).
2. Used space: 8.6 GB.
3. Available space: 10 GB.
4. Percentage utilization: 47%.
5. Space occupied by home directory (/root): 60 KB.
6. Space occupied by cyber_lab directory: 32 KB.

Difference between df and du: df ("disk free") reports space usage at the filesystem/mount level — total size, used, and available space for an entire mounted partition. du ("disk usage") reports space consumed by specific files and directories, letting you drill down into what inside a filesystem is actually taking up the space. In short, df answers 'how full is this filesystem?' while du answers 'how much space does this folder/file use?' — and the two can appear to disagree slightly because df also accounts for filesystem overhead and space held by deleted-but-open files.

Exercise 17 – Cybersecurity Investigation Scenario

Scenario: investigating a Linux machine after suspicious activity has been reported, examining only the filesystem.

Location	Investigate?	Why / expected security-relevant information
/home	Yes	User-created files, downloaded tools, shell history, and personal .bash_history/.ssh keys can show what an account did or whether it was used to stage an attack.
/etc	Yes	Configuration changes reveal privilege escalation or persistence: new/modified users in passwd, added entries in sudoers, altered cron jobs, or tampered hosts/DNS settings.
/var/log	Yes	Primary evidence source: auth logs, package install/removal history, and service logs show login attempts, commands run as root, and a timeline of events.
/tmp	Yes	World-writable and commonly abused to stage payloads, downloaded exploit tools, or temporary exfiltration data; file owners and timestamps help build a timeline.
/proc	Yes	Shows live/running processes, open network sockets, and command lines at the moment of investigation — critical for catching an active malicious process before it's killed or a reboot erases it.
/dev	Limited	Normally just device nodes; worth a quick check for unusual devices or files masquerading as device nodes, but lower priority than the others unless something looks off.
/root	Yes	The administrator's home directory; unauthorized files, altered shell configs (.bashrc), or new SSH keys here would indicate the attacker gained root-level access.